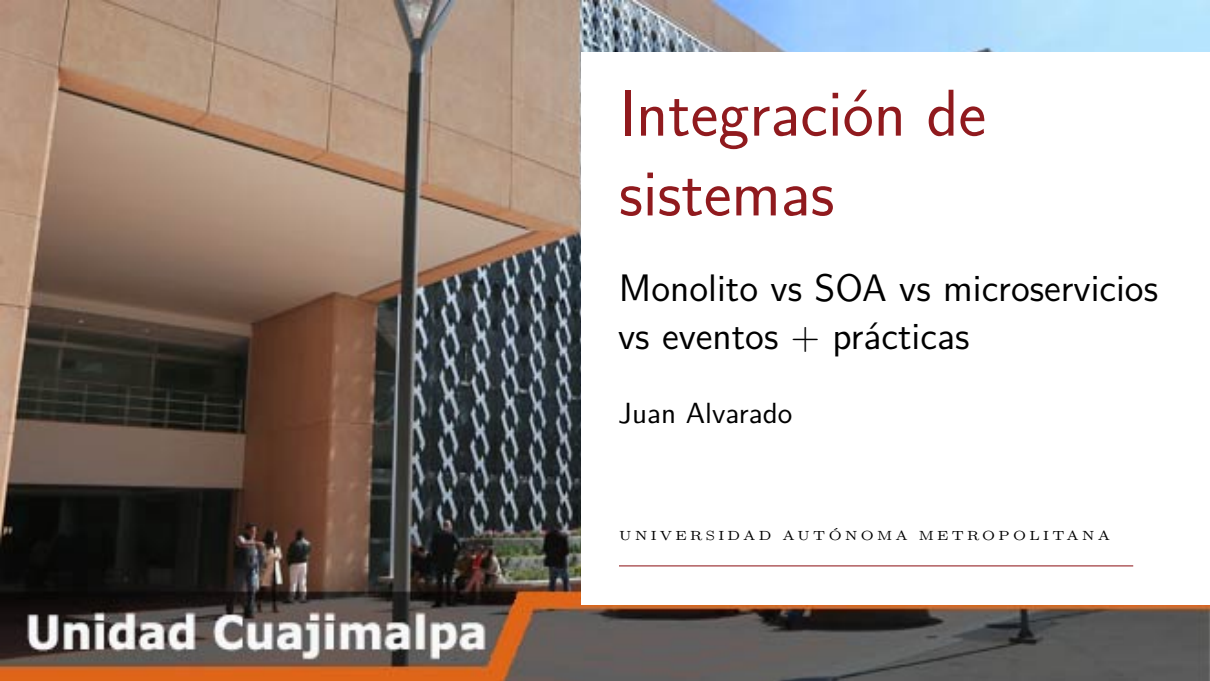




Integración de sistemas

Monolito vs SOA vs microservicios
vs eventos + prácticas

Juan Alvarado

UNIVERSIDAD AUTÓNOMA METROPOLITANA

Unidad Cuajimalpa

Monolito: una sola aplicación

Monolito: una sola aplicación

Monolito: una sola aplicación

Un monolito es una aplicación donde la mayor parte de la lógica de negocio, la interfaz y el acceso a datos se despliegan como una sola unidad.

Es sencillo de desarrollar y desplegar al inicio: un solo proyecto, una base de código y un artefacto de despliegue.



Monolito: una sola aplicación

El escalamiento suele ser en toda la aplicación: se clona el monolito completo en más servidores o contenedores.

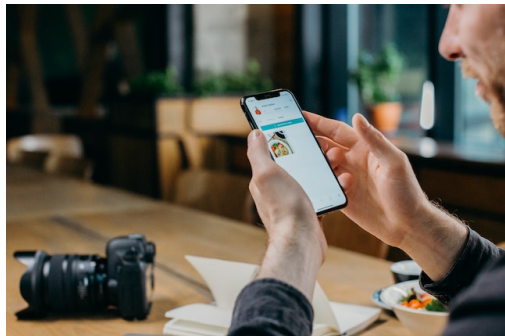


Monolito: una sola aplicación

Con el tiempo, a medida que el código crece, se vuelve más difícil de entender y de modificar sin introducir errores, especialmente si no hay una modularidad interna clara.

Monolito: una sola aplicación

Aun así, para sistemas pequeños o de baja complejidad, un monolito bien estructurado puede ser una solución muy razonable.



Monolito: una sola aplicación

Actividad – Diagnóstico de un monolito

En equipos, piensen en un sistema que conozcan o usen (por ejemplo, un ERP, un LMS de la universidad, una app interna de alguna empresa).

Discutiran unos minutos y decidir si sospechan que es monolítico o no, y por qué

Monolito: una sola aplicación

- ¿Se despliega todo como un solo paquete?
- ¿Qué pasaría si se cae “una parte”?
- ¿Se cae todo?

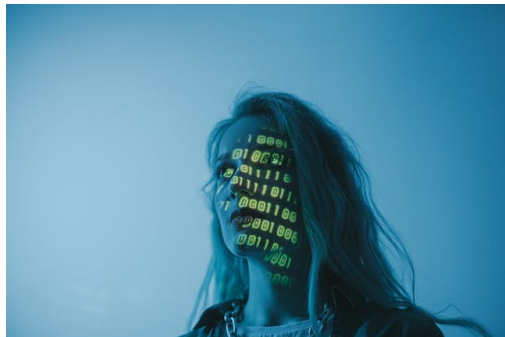


SOA en contraste con el monolito

SOA en contraste con el monolito

SOA en contraste con el monolito

En contraste, SOA divide la funcionalidad en servicios más grandes y reutilizables que se exponen a través de interfaces bien definidas, a menudo usando estándares como SOAP.



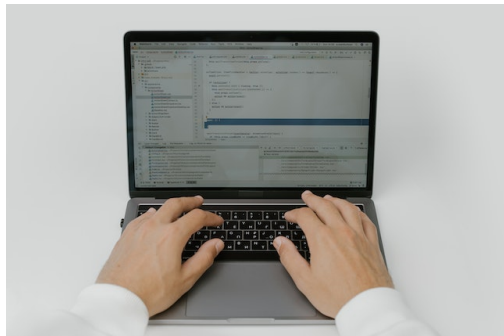
SOA en contraste con el monolito

Estos servicios pueden ser consumidos por múltiples aplicaciones, lo que ayuda a evitar duplicar lógica de negocio (por ejemplo, reglas de facturación) en varios sistemas.

SOA en contraste con el monolito

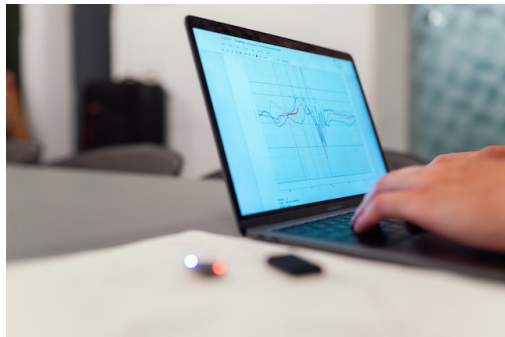
SOA suele apoyarse en un ESB u otra infraestructura central para orquestar flujos de trabajo y transformar mensajes entre formatos distintos.

La gobernanza (versionado de servicios, seguridad, monitoreo) se vuelve un tema clave en este enfoque.



SOA en contraste con el monolito

Aunque el objetivo sea modularizar, en la práctica puede formarse un “monolito distribuido” si todas las dependencias pasan por un único bus y cambian al mismo ritmo.



Microservicios: servicios pequeños e independientes

Microservicios: servicios pequeños e
independientes

Microservicios: servicios pequeños e independientes

Los microservicios llevan la idea de servicios un paso más allá, proponiendo dividir la aplicación en muchos servicios pequeños, cada uno con una responsabilidad acotada y despliegue independiente.

Microservicios: servicios pequeños e independientes

Cada microservicio se puede implementar en tecnologías distintas, siempre que cumpla su contrato de API, lo que da flexibilidad tecnológica.

Microservicios: servicios pequeños e independientes

El escalamiento puede hacerse solo en los servicios que lo necesiten (por ejemplo, escalar Pedidos sin escalar Catálogo), lo cual mejora el uso de recursos.



Microservicios: servicios pequeños e independientes

Sin embargo, la complejidad operacional aumenta: se necesitan prácticas sólidas de DevOps, monitoreo distribuido, descubrimiento de servicios y manejo de fallos entre servicios.

Microservicios: servicios pequeños e independientes

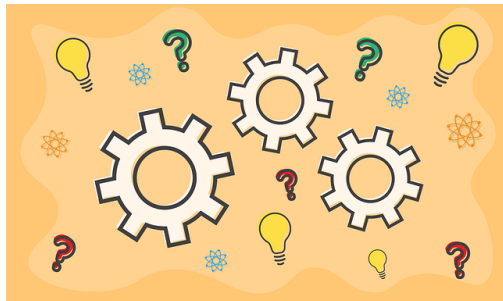
Microservicios son más adecuados cuando se espera un crecimiento fuerte, equipos autónomos y una necesidad de desplegar cambios en partes específicas del sistema con alta frecuencia.

Microservicios: servicios pequeños e independientes

Actividad – Descomponer un módulo en microservicios

Consideren un escenario de e-commerce, pero en lugar de pensar en toda la tienda, elijan un área concreta (por ejemplo, “Gestión de pedidos”).

En equipos, definan la posible separación en microservicios:

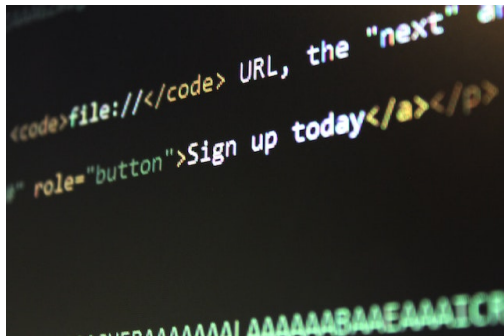


Microservicios: servicios pequeños e independientes

- Servicio de Carrito.
- Servicio de Pedidos.
- Servicio de Pagos.
- Servicio de Notificaciones.

Microservicios: servicios pequeños e independientes

Cada equipo debe justificar brevemente por qué separaría así y qué ventajas ve.



Arquitecturas dirigidas por eventos (event-driven)

Arquitecturas dirigidas por eventos
(event-driven)

Arquitecturas dirigidas por eventos (event-driven)

La arquitectura dirigida por eventos se basa en que los componentes emiten eventos cuando ocurre algo relevante (por ejemplo, "PedidoCreado") y otros componentes se suscriben para reaccionar.

Arquitecturas dirigidas por eventos (event-driven)

En lugar de hacer llamadas sincrónicas directas (por ejemplo, REST) entre servicios, se envían mensajes a un broker o cola de eventos, lo que desacopla productores y consumidores.



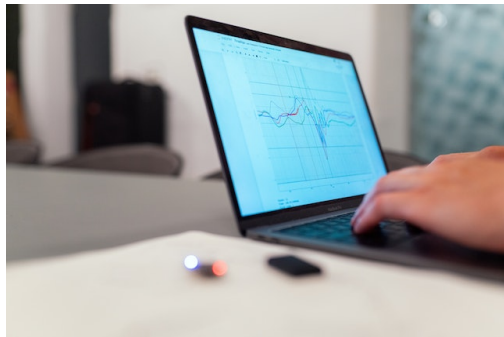
Arquitecturas dirigidas por eventos (event-driven)

Este desacoplamiento facilita agregar nuevos consumidores sin modificar el emisor original (por ejemplo, un nuevo servicio de analítica que escucha los eventos de pedidos).



Arquitecturas dirigidas por eventos (event-driven)

Las arquitecturas basadas en eventos se integran bien con microservicios, ya que permiten que cada servicio mantenga su propio modelo de datos y se sincronice con el resto a través de eventos.



Arquitecturas dirigidas por eventos (event-driven)

No obstante, la visibilidad de los flujos puede volverse compleja y el manejo de consistencia eventual requiere diseño cuidadoso.



Arquitecturas dirigidas por eventos (event-driven)

Actividad – Tormenta de eventos

En equipos, definan eventos clave para el e-commerce: e.g. “CarritoCreado”, “ProductoAgregado”, “PedidoPagado”, “PedidoEnviado”, etc.



Arquitecturas dirigidas por eventos (event-driven)

En equipos, elijan alguno(s) de los eventos definidos y respondan:

- ¿Qué servicios podrían emitirlo?
- ¿Qué servicios podrían suscribirse y qué acciones dispararían?

Comparación conceptual: monolito, SOA, microservicios, eventos

Comparación conceptual: monolito, SOA,
microservicios, eventos

Comparación conceptual: monolito, SOA, microservicios, eventos

A nivel conceptual, estos estilos se pueden ver como etapas y opciones en la evolución de la integración, más que como “versiones” que siempre se reemplazan.

- Monolito: enfoque simple, todo junto; bueno para empezar o para sistemas pequeños.



Comparación conceptual: monolito, SOA, microservicios, eventos

- SOA: modulariza en servicios grandes, suele apoyarse en estándares de servicios web y middleware central.
- Microservicios: servicios pequeños, independientes, con alto grado de autonomía y despliegue independiente.



Comparación conceptual: monolito, SOA, microservicios, eventos

- Eventos: una forma de integrar componentes basada en mensajes y suscripción, más que en llamadas directas.

Comparación conceptual: monolito, SOA, microservicios, eventos

En la práctica, es común combinar enfoques; por ejemplo, un monolito que expone APIs y se integra con otros sistemas mediante eventos o servicios SOAP heredados.

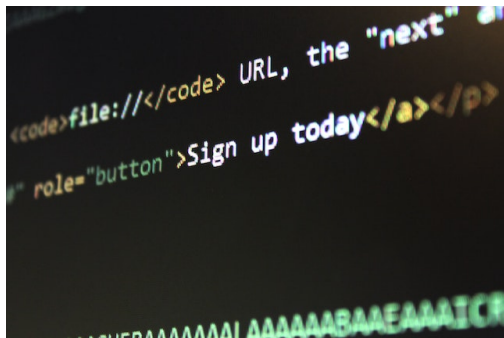


Comparación conceptual: monolito, SOA, microservicios, eventos

Actividad – Matriz de decisión

Considérense los siguientes criterios:

- Tamaño del equipo.
- Velocidad de cambio.
- Regulación/seguridad.
- Presupuesto de operación.



Comparación conceptual: monolito, SOA, microservicios, eventos

Para cada criterio, dar ejemplos de cuándo sería razonable preferir monolito, SOA, microservicios o eventos.

[illegible]

Práctica: observar integración monolítica vs API REST simple

Práctica: observar integración monolítica
vs API REST simple

Práctica: observar integración monolítica vs API REST simple

Consultar el archivo de la práctica

Probar la API y observar su naturaleza monolítica

- ¿Qué responsabilidades están juntas en `index.js`?
- ¿Qué pasaría si el módulo de pedidos necesitara escalar más que el de productos?



Práctica: observar integración monolítica vs API REST simple

Esbozar una posible partición en servicios

Sin implementar nada extra, cada equipo propone en el pizarrón:

- Qué partes de la lógica separarían en servicios (por ejemplo, Servicio de Productos, Servicio de Pedidos).



Para llevar ...

Para llevar ...

Para llevar ...

- Cómo evolucionó la integración desde conexiones punto a punto, pasando por SOA/SOAP, hasta REST y estilos más recientes.
- Qué características distinguen a monolito, SOA, microservicios y arquitecturas dirigidas por eventos.



Para llevar ...

- Qué observaron en las prácticas sobre responsabilidades concentradas vs separadas.
- Qué pueden opinar, sin haber cubierto con detalle el tema, sobre llamadas directas vs publicación de eventos.

¡Gracias!

Unidad Cuajimalpa



Unidad Cuajimalpa